

Project Kessler: Autonomous Operations Intelligence Desk (AOID)

Project Kessler: Autonomous Operations Intelligence Desk (AOID)

Hackathon Target: Software Track — Problem Statement 5 (Autonomous Operations Intelligence Desk)

1. Summary & Core Concept

Problem: Existing ground-based collision warnings rely on imprecise radar data with massive uncertainty bounds, known as "error ellipsoids". As a result, satellites execute premature, high- Δv avoidance maneuvers for false positives, burning critical fuel and shortening their operational lifespan.

Solution (PS 5 Alignment): An autonomous operations intelligence desk that correlates ground-station predictions with real-time edge-vision computing. This architecture repurposes existing onboard Star Trackers and offloads image processing to a dedicated software edge-process, preserving primary flight computer resources.

Operational Workflow:

1. **Coarse Warning Ingestion:** The satellite receives ground-station Conjunction Data Messages (CDMs) flagging potential high-risk conjunctions weeks in advance.
2. **Predictive Alignment:** The spacecraft uses reaction wheels to slew its orientation, pointing an existing Star Tracker along the predicted threat approach vector during non-mission orbits.
3. **Parallel Edge Processing:** The raw Star Tracker video stream is tapped into an independent, resource-constrained software process (C++/Python) to prevent

hijacking the primary Attitude Determination and Control System (ADCS).

4. **Transient Streak Extraction:** The edge process runs real-time frame differencing to strip static background stars and isolate high-velocity debris streaks.
5. **Intersection Verification:** The system checks if the computed 3D trajectory intersects the satellite's defined safety volume.
6. **Proactive Burn Execution:** If the collision probability exceeds the threshold, the satellite calculates and executes a minimal Δv maneuver at the optimal orbital node long before closest approach.

2. Active-Active Avoidance Protocol (Spacecraft vs. Spacecraft)

When a predicted conjunction involves two fully functional, maneuverable satellites, executing double maneuvers escalates the collision risk. The onboard system runs a multi-variable decision engine to determine which spacecraft executes the maneuver.

Decision Matrix & Scoring Function: Both spacecraft exchange minimal state packets via ground uplink or inter-satellite links. The system calculates a Maneuver Responsibility Score (MRS) based on four weighted metrics:

$$MRS = w_1 \cdot \text{Cost}_{\text{lifetime}} + w_2 \cdot \text{Cost}_{\Delta v} + w_3 \cdot \text{Cost}_{\text{priority}} + w_4 \cdot \text{Cost}_{\text{downtime}}$$

The satellite with the lower score performs the maneuver:

1. **Remaining Delta-V Reserves:** The satellite with greater remaining fuel reserves assumes priority to execute the burn.
2. **Remaining Operational Lifetime:** A satellite near End-of-Life (EOL) preserves its remaining fuel; the younger asset with a longer operational margin takes the burn.
3. **Mission Priority / Payload Criticality:** High-value national defense or primary scientific payloads are prioritized to remain on track over secondary or commercial payloads.

4. **Operational Downtime / Recovery Latency:** Measures the time required for a satellite to re-orient, re-stabilize, and resume revenue/mission operations post-burn.

3. Ground-Station AI Integration (Drag Prediction)

The Objective: Predict atmospheric drag fluctuations to tighten the error ellipsoid before uplinking the data to the satellite.

The Stack: The model is extremely lightweight. The architecture utilizes a standard XGBoost regressor or a simple LSTM time-series model in Python to prevent the overfitting and training latency associated with massive Deep Neural Networks.

The Inputs (Features):

- **Space Weather Data:** Pulls the F10.7 solar flux and Kp index (geomagnetic activity) APIs from NOAA. Solar flares expand the atmosphere and cause sudden drag.
- **Historical SSA Data:** Pulls historical Two-Line Elements (TLEs) from Space-Track.org to track how specific debris orbits have degraded in the past.

The Output: The model generates a predictive drag coefficient, which is used to calculate a refined Conjunction Data Message (CDM) with a significantly smaller error margin.

4. Division of Labor (4-Person Software Architecture)

This architecture is designed so that no one is blocked by anyone else. Each person builds their component as an isolated node communicating via strictly typed JSON packets over local TCP.

Role 1: The Systems Architect (Physics & Dynamics)

- **Task 1 (Truth Model):** Write the Basilisk C++/Python script to propagate spacecraft and debris orbits, configuring the time-reversed math to guarantee a Time of Closest Approach (TCA) collision geometry.
- **Task 2 (Flight Software):** Implement the `mrpFeedback` closed-loop attitude control system to slew the spacecraft and lock the Star Tracker field-of-view

onto the threat vector.

- **Task 3 (Telemetry Router):** Hook Basilisk to Vizard for 3D rendering and stream the synthetic optical frames and 6-DOF state telemetry to the local TCP socket.

Role 2: Edge Vision Engineer (Embedded Processing)

- **Task 1 (Ingestion):** Write a lightweight C++ or Python (OpenCV) edge script that subscribes to the raw video frame TCP socket.
- **Task 2 (Frame Differencing):** Write the memory-constrained logic to subtract the previous frame from the current frame to erase static background stars.
- **Task 3 (Extraction):** Output a strict JSON packet containing the \$(X, Y)\$ pixel coordinates of the detected debris streak centroid.

Role 3: Machine Learning Engineer (Ground AI)

- **Task 1 (Data Fetching):** Write a script to continuously pull the NOAA F10.7 solar flux API, Kp index, and Space-Track.org TLEs.
- **Task 2 (The Model):** Train the XGBoost or LSTM model to correlate solar weather with atmospheric drag.
- **Task 3 (The Output):** Stream the resulting refined error ellipsoid data as JSON to the shared dashboard.

Role 4: Mission Control Dashboard & Integration (PS 5 UI)

- **Task 1 (The Operations Desk):** Build a unified Streamlit dashboard mimicking a military/aerospace intelligence desk.
- **Task 2 (Data Ingestion):** Parse the JSON payloads from the Ground AI, Physics Engine, and Edge Vision nodes dynamically.
- **Task 3 (Visualization & Audit):** Render live metric cards (latency, remaining fuel), the side-by-side raw vs. processed optical frames, and an auditable event log of the Active-Active decision matrix executing the maneuver.

5. Development Roadmap (48-Hour Phasing)

Phase 1: Physics & Environment Setup

- Initialize the Basilisk 2-body orbital propagator and central Earth gravity environment.
- Execute backward numerical integration to define exact conjunction state vectors.
- Define the shared JSON packet schemas in the `shared/` directory to unblock all team members.

Phase 2: Closed-Loop ADCS & Optical Rendering

- Model reaction wheel torque and spacecraft slew rates.
- Connect `simpleNav`, `inertial3D`, and `mrpFeedback` to orient the $15^\circ \times 15^\circ$ Star Tracker Field of View.
- Synthesize optical frame outputs via Vizard, replicating sensor resolution, dark current noise, and moving debris streaks.

Phase 3: Edge Vision Software Node

- Initialize the local TCP loopback.
- Implement parallel line-buffer memory structures for high-speed background subtraction in software.
- Extract the streak vectors and route the detection flags back to the main pipeline.

Phase 4: AOID Dashboard Integration & Validation

- Connect the Streamlit UI to the live socket feeds.
- Code the multi-variable scoring model (\$\$\$) for operational satellite-to-satellite burn negotiation.
- Calculate the distance of closest approach (DCA) and collision probability against the spacecraft's safety bounding box to trigger the final UI alerts.